

## 实验 3 复杂查询和触发器

班级：07112304

学号：1120233329

姓名：陈墨霏

### 一、实验目的

掌握 SQL 嵌套查询和集合查询等各种高级查询的设计方法；掌握数据库触发器的设计和使用方法。

### 二、实验内容

#### 1. 请用 SQL 语言实现以下查询

- (1) 查询订购了“徐州市泰力公司矿山设备四厂”制造的“活塞式减压阀”的顾客的信息。
- (2) 查询没有购买过“徐州市泰力公司矿山设备四厂”制造的“活塞式减压阀”的顾客的信息。
- (3) 查询订单平均金额超过 500 元的顾客中的中国籍顾客的信息。
- (4) 查询顾客“薛融”订购过而“宣荣揣”没有订购过的零件的信息。

请将查询结果截图附在实验报告中（可能会有查询结果为空，但也不一定）

#### 2. 请建立以下触发器

- (1) 在 Lineitem 表上定义一个 INSERT 触发器，当增加一项订单明细时，自动修改订单表 Orders 中的订单总金额，以保持数据的一致性。（增加的那一部分金额为：零售价×数量×折扣）
- (2) 在 Lineitem 表上定义一个 BEFORE UPDATE 触发器，当修改订单明细中的数量时，先检查零件供应表 PartSupp 中的可用数量是否足够。

触发器建好之后，请进行实验以验证触发器建立是否正确，并将结果截图和必要的文字解释附在实验报告中。

### 三、实验步骤

#### 3.1 SQL 语句实现复杂查询

发起查询请求前，注意先选择架构：

```
USE tpch;
```

##### 3.1.1 查询请求一

|                                        |
|----------------------------------------|
| 查询订购了“徐州市泰力公司矿山设备四厂”制造的“活塞式减压阀”的顾客的信息。 |
|----------------------------------------|

## 题目分析

### 编写查询语句

```
SELECT DISTINCT c.*  
FROM customer c  
JOIN orders o ON o.customer_id = c.customer_id  
JOIN lineitem l ON l.order_id = o.order_id  
JOIN supplier s ON s.supplier_id = l.supplier_id  
JOIN part p ON p.part_id = l.part_id  
WHERE supplier_name = '徐州市泰力公司矿山设备四厂'  
AND part_name = '活塞式减压阀';
```

#### 3.1.2 查询请求二

查询没有购买过“徐州市泰力公司矿山设备四厂”制造的“活塞式减压阀”的顾客的信息。

## 题目分析

### 编写查询语句

#### 3.1.3 查询请求三

查询订单平均金额超过 500 元的顾客中的中国籍顾客的信息。

## 题目分析

### 编写查询语句

#### 3.1.4 查询请求四

查询顾客“薛融”订购过而“宣荣瑞”没有订购过的零件的信息。

## 题目分析

## 编写查询语句

### 3.2 触发器的建立

#### 3.2.1 建立 INSERT 触发器 trig-update-orders-total-amount

在 Lineitem 表上定义一个 INSERT 触发器，当增加一项订单明细时，自动修改订单表 Orders 中的订单总金额，以保持数据的一致性。（增加的那一部分金额为：零售价×数量×折扣）

## 题目分析

触发器的作用是当在 `lineitem` 表中插入新记录时，自动更新 `orders` 表中相对应的 `total_amount` 字段。具体步骤如下：

1. 检查插入的 `part_id` 是否在 `part` 表中存在，如果不存在，则抛出异常。
2. 如果 `part_id` 存在，则获取该零件的零售价。
3. 检查 `orders` 表中是否存在对应的 `order_id`，如果不存在，则插入一条新的订单记录，初始 `total_amount` 为 0。
4. 更新 `orders` 表中对应的 `total_amount`，计算公式为：零售价 × 数量 × 折扣。

## 触发器创建 SQL 语句和说明

```
DELIMITER $$

CREATE TRIGGER trig_update_orders_total_amount
AFTER INSERT ON lineitem
FOR EACH ROW
BEGIN
    DECLARE v_price DECIMAL(10,2);
    DECLARE v_exists INT;

    -- 判断 part 是否存在
    SELECT COUNT(*) INTO v_exists
    FROM part
    WHERE part_id = NEW.part_id;
```

```
-- 如果 part 不存在, 则抛出异常
IF v_exists = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Invalid part_id: part does not exist';
END IF;

-- 获取零售价
SELECT retail_price INTO v_price
FROM part
WHERE part_id = NEW.part_id;

-- 判断 orders 是否存在
SELECT COUNT(*) INTO v_exists
FROM orders
WHERE order_id = NEW.order_id;

IF v_exists = 0 THEN
    -- 如果订单不存在, 则插入新订单记录 (初始 total_amount = 0)
    INSERT INTO orders(order_id, total_amount)
    VALUES (NEW.order_id, 0);
END IF;

-- 更新 total_amount
UPDATE orders
SET total_amount = total_amount + (v_price * NEW.quantity * NEW.discount)
WHERE order_id = NEW.order_id;
END$$

DELIMITER ;
```

几点说明（课堂上没有提到过的语法）：

1. **DECLARE**：用于声明变量。这里声明了两个变量 `v_price` 和 `v_exists`，分别用于存储零售价和判断记录是否存在的标志。

2. **SELECT COUNT(\*) INTO v\_exists**: 用于查询记录的数量, 并使用 **INTO** 将结果存储到变量 **v\_exists** 中。
3. **DELIMITER \$\$**: 用于改变 SQL 语句的结束符, 以便在触发器中使用分号。在默认情况下, MySQL 使用分号 **;** 作为 SQL 语句的结束符号。但是——在创建存储过程、触发器、函数等复杂结构时, 代码中会包含多个分号, 这会导致 MySQL 误认为 SQL 语句结束了, 而引发错误。因此我们需要临时改变结束符号为 **\$\$**, 以便 MySQL 正确解析整个触发器的定义。在触发器定义结束后, 我们再将结束符号改回分号 **;**。
4. **SIGNAL SQLSTATE '45000'**: 用于抛出自定义异常。**45000** 是一个通用的 SQLSTATE 错误代码, 表示用户定义的异常。可以根据需要自定义错误消息。MySQL 不支持 **RAISE EXCEPTION** 语法, 因此使用 **SIGNAL** 来抛出异常。

### 3.2.2 建立 BEFORE UPDATE 触发器 trig-check-avail-qty

在 Lineitem 表上定义一个 BEFORE UPDATE 触发器, 当修改订单明细中的数量时, 先检查零件供应表 PartSupp 中的可用数量是否足够。

## 四、实验结果及分析

## 五、实验收获与体会

## 附录：程序清单及说明

以下是本实验中涉及的主要程序文件及其说明：

1. **search.sql**: 包含了所有的查询请求和对应的 SQL 语句。
2. **trigger.sql**: 包含了所有的触发器创建语句。

以上文件均已在实验中详细说明，并附有必要的注释以便理解和复现实验过程。